CSE 4513

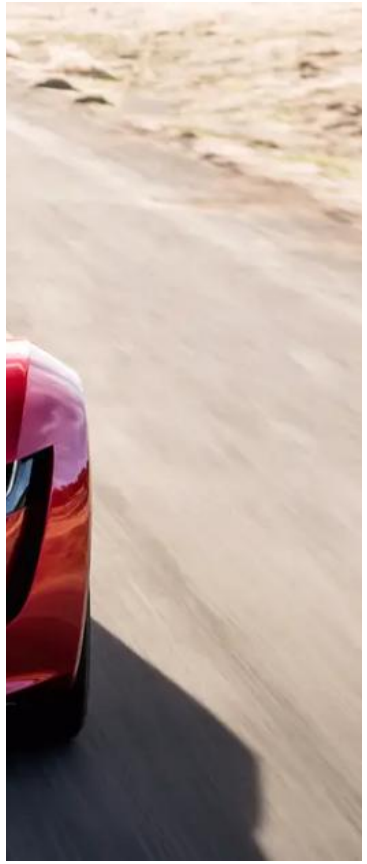
Lec – 4

Requirements Engineering

MISLEADING REQUIREMENTS

Father: What do you want from me on your birthday?

Son: Something that moves from 0 to 60 within 3 seconds.



Father's Interpretation

Expected Delivered to Son

MISLEADING REQUIREMENTS

Customer Requirements:

1. Have one trunk
2. Have four legs
3. Should carry load both passenger & cargo
4. Blackish in color
5. Should be herbivorous

Our Solution:

1. Have one trunk ✓
2. Have four legs ✓
3. Should carry load both passenger & carg ✓
4. Black in color ✓
5. Should be herbivorous ✓

Our Value addition: **Also gives**

BACKGROUND..

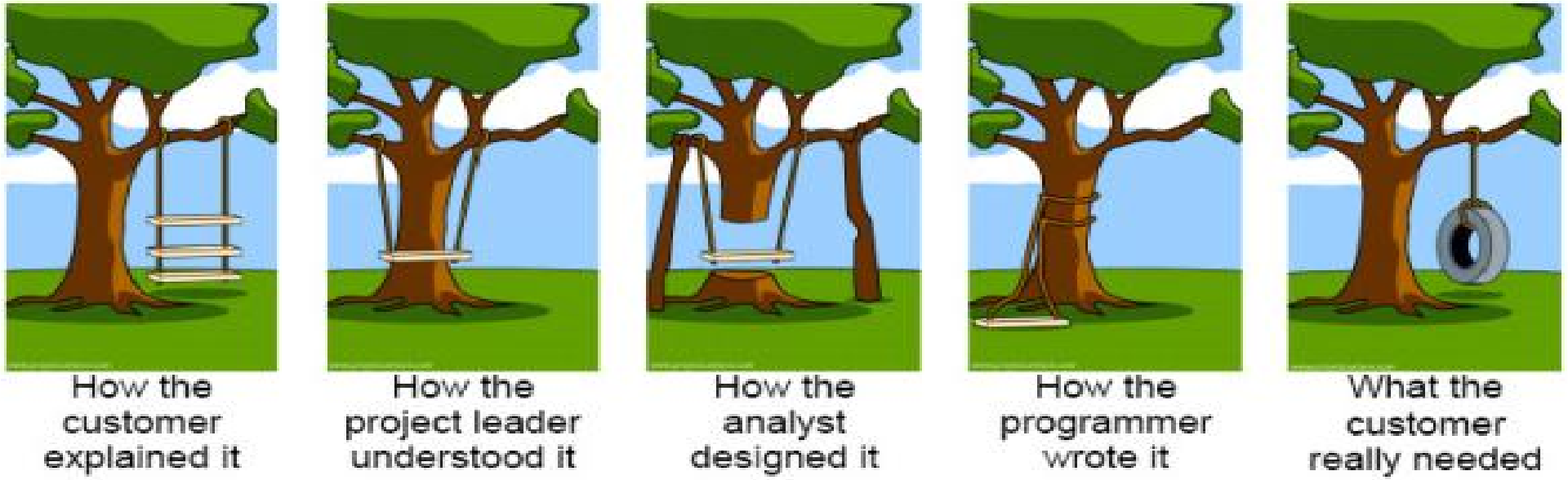
- ✓ What is a Requirement?
  - ➢ A condition or capability that must be possessed by a system (IEEE)
- ✓ The requirements task:
  - ➢ Input: User needs in minds of people
  - ➢ Output: precise statement of what the future system will do
- ✓ What is the output of the Req. phase ?
  - ➢ A software requirements specification (SRS) document
- ✓ What is an SRS ?
  - ➢ A complete specification of what the proposed system should do!

BACKGROUND..

- ✓ Requirements understanding is hard
  - ➢ Visualizing a future system is difficult
  - ➢ Capability of the future system not clear, hence needs are not clear
  - ➢ Requirements change with time
  - ➢ …
- ✓ It is essential to do a proper analysis and specification of requirements

BACKGROUND..

- ✓ more common problems with requirements definition are :
  1. Requirements do not represent the actual needs of the customer.
  2. Requirements are incomplete or conflicting.
  3. Requirements are difficult and expensive to update after they have been agreed upon.
  4. Customers, requirements analysts, and software engineers who develop the system have difficulties understanding each other.



REQUIREMENT ENGINEERING

- ✓ The **process** of **establishing the services** that the customer requires from a system **and** the **constraints** under which it operates and is developed
- ✓ The requirements themselves are the **descriptions of the system services and constraints** that are generated during the requirements engineering process.

Types of Requirements

User requirements

- ✓ Statements in natural language plus diagrams of the services the system provides and its operational constraints.
- ✓ Written for customers.

System requirements

- ✓ A structured document setting out detailed descriptions of the system's functions, services and operational constraints

User Requirement

The MHC-PMS shall generate monthly management reports showing the cost of drugs prescribed by each clinic during that month.

System Requirements

- ✓ On the last working day of each month, a summary of the drugs prescribed, their cost, and the prescribing clinics shall be generated.
- ✓ The system shall automatically generate the report for printing after 17.30 on the last working day of the month.
- ✓ If drugs are available in different dose units (e.g., 10 mg, 20 mg) separate reports shall be created for each dose unit.

FUNCTIONAL AND NON-FUNCTIONAL REQUIREMENTS

Functional requirements

- ✓ Describe functionality or system services.
- ✓ Depend on the type of software, expected users and the type of system where the software is used.
- ✓ Functional user requirements may be high-level statements of what the system should do.
- ✓ Functional system requirements should describe the system services in detail.

FUNCTIONAL AND NON-FUNCTIONAL REQUIREMENTS

Non-functional requirements

- ✓ These define system properties and constraints e.g. reliability, response time, maintainability, scalability, portability, and storage requirements.
- ✓ Constraints are I/O device capability, system representations, etc.

- ✓ Process requirements may also be specified mandating a particular IDE, programming language or development method.
- ✓ Non-functional requirements may be more critical than functional requirements. If these are not met, the system may be useless
- ✓ Non-functional requirements may affect the overall architecture of a system rather than the individual components.

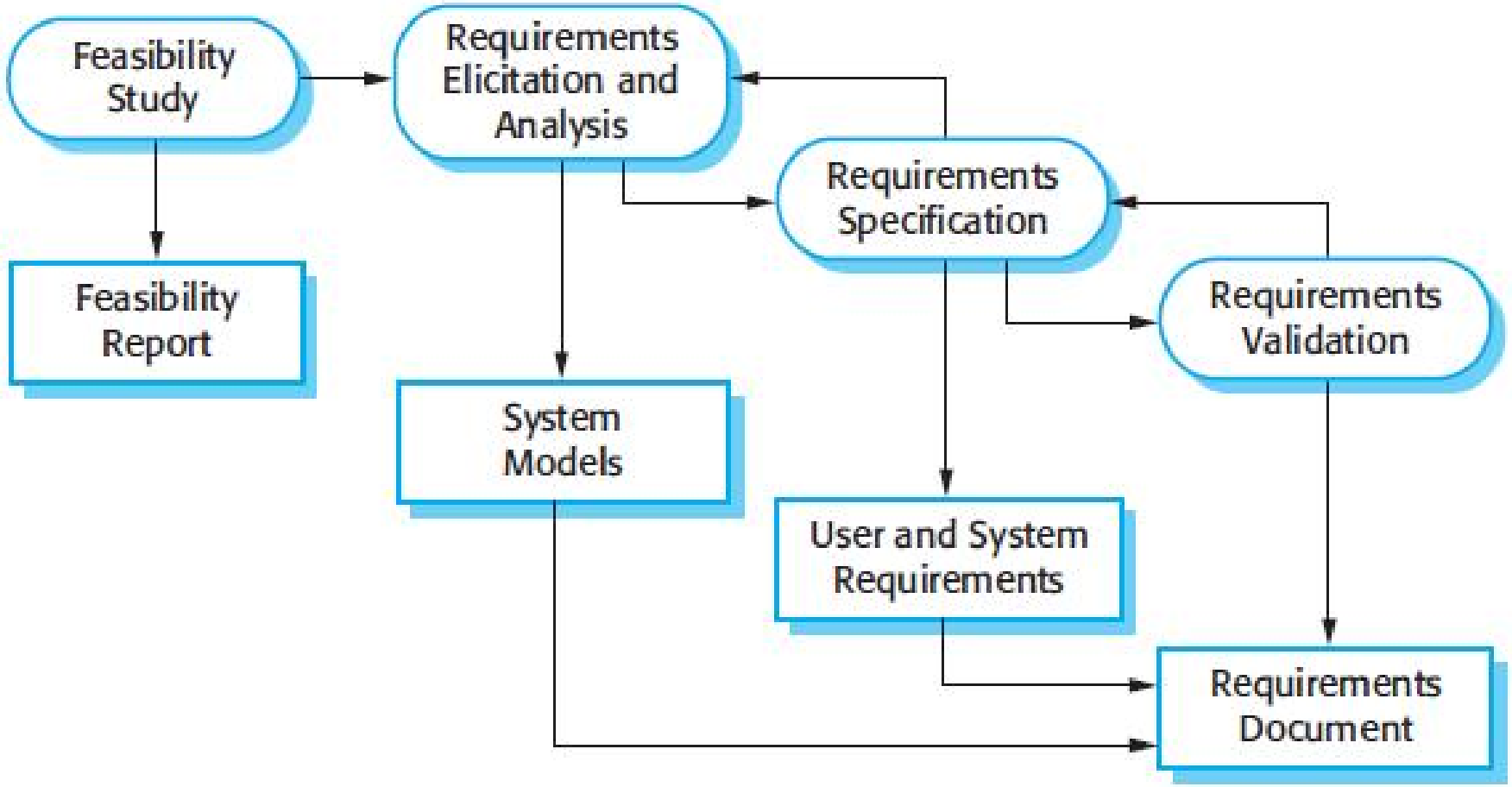
FUNCTIONAL AND NON-FUNCTIONAL REQUIREMENTS

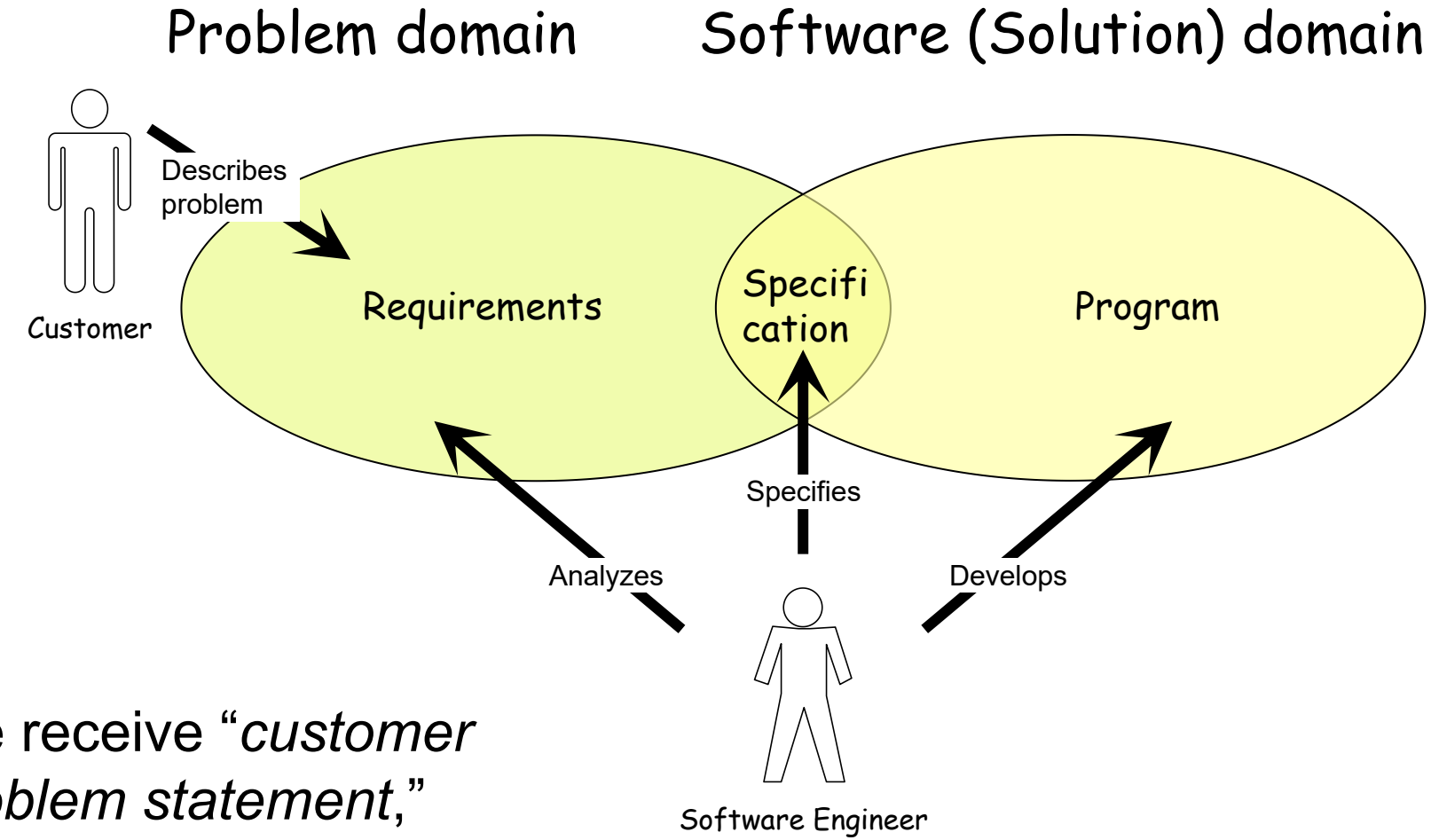
Non-functional requirements

Property	Measure
Speed	Processed transactions/second User/event response time Screen refresh time
Size	Mbytes Number of ROM chips
Ease of use	Training time Number of help frames
Reliability	**Mean time to failure (MTTF)** Probability of unavailability Rate of failure occurrence Availability
Robustness	Time to restart after failure **(MTTR)** Percentage of events causing failure Probability of data corruption on failure
Portability	Percentage of target dependent statements Number of target systems

Metrics for specifying nonfunctional requirements

REQUIREMENTS ENGINEERING PROCESS

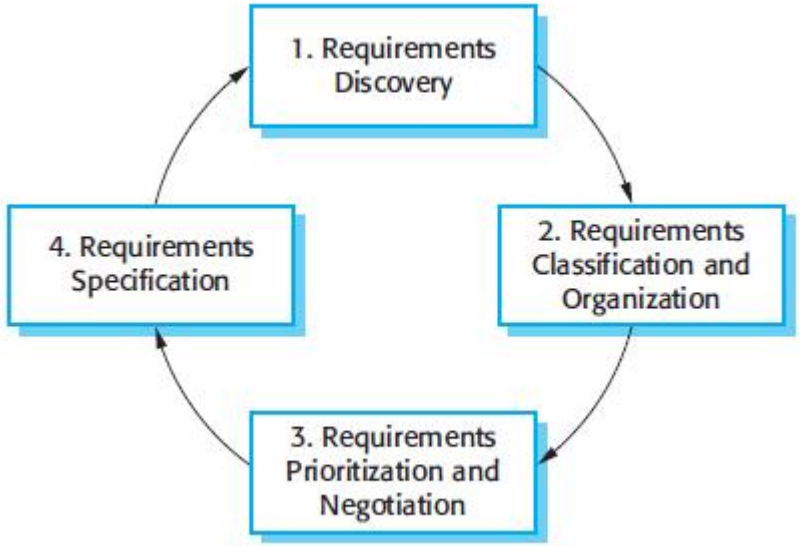




We receive "*customer problem statement*,"
not the requirements!

REQUIREMENTS ENGINEERING PROCESS

Requirements elicitation and analysis process



Requirements discovery is the process of interacting with stakeholders of the system to discover their requirements.

Requirements classification and organization takes the unstructured collection of requirements, groups related requirements, and organizes them into coherent clusters.

Requirements prioritization and negotiation takes place when multiple stakeholders are involved, requirements will conflict. This activity is concerned with prioritizing requirements and finding and resolving requirements conflicts through negotiation.

Requirements specification is the process of documenting all the requirements and input into the next round.

REQUIREMENTS ENGINEERING PROCESS

Requirements Validation

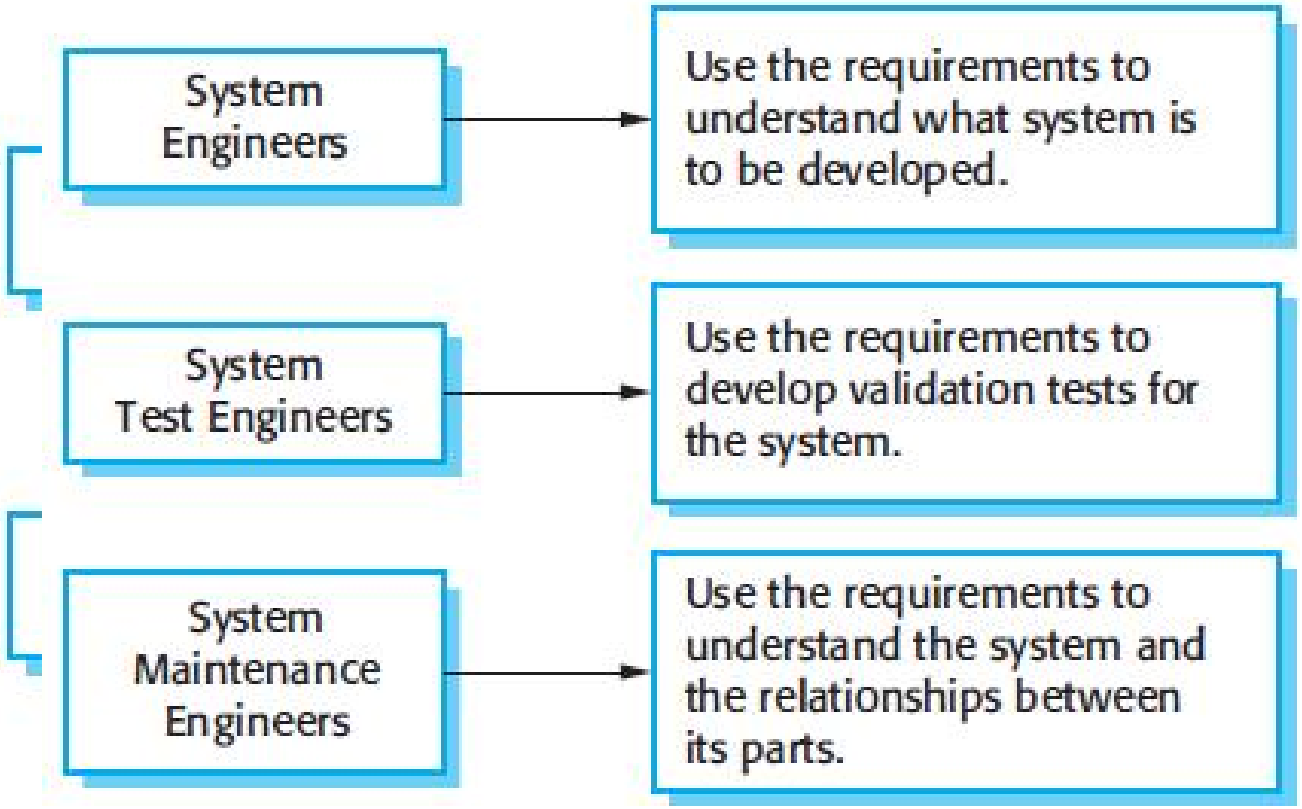
- ✓ is the process of interacting with stakeholders of the system to discover their requirements.
- ✓ During the requirements validation process, different types of checks should be carried out on the requirements in the requirements document.
  - ✓ *Validity checks*
  - ✓ *Consistency checks*
  - ✓ *Completeness checks*
  - ✓ *Realism checks*
  - ✓ *Verifiability*

If a requirement states that the system should load a page for a user requests it and to send notifications to mobile devices, the validity check would confirm that the feature supports all relevant mobile platforms the user expect, like Android and iOS.

If a requirement states that the system should load a page for a user requests it and to send notifications to mobile devices, the verifiability check would ensure that this requirement is stated in measurable terms, such as "the page should load within 2 seconds under typical network conditions.

SOFTWARE REQUIREMENTS DOCUMENT

- ✓ Called the software requirements specification or SRS
- ✓ an official statement of what the system developers should implement.
- ✓ include both the user requirements for a system and a detailed specification of the system requirements.
- ✓ The requirements document has a diverse set of users



COMPONENTS OF AN SRS

- ✓ An SRS must specify following types of requirements:
  - ➢ Functional
  - ➢ Performance (sometime mentioned as non-functional requirement)
  - ➢ Design constraints
  - ➢ External interfaces

COMPONENTS OF AN SRS

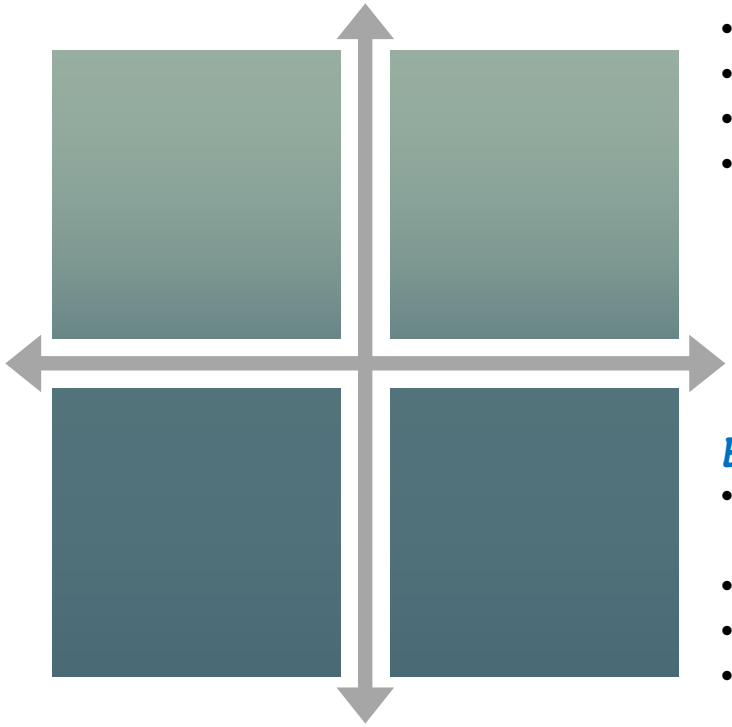
Types of Requirements

Functional

- Specifies all the functionality that the system should support
- Outputs for the given inputs and the relationship between them
- All operations the system is to do
- Must specify behavior for invalid inputs too

Non-functional

- All the performance constraints on the software system
- Generally on response time , throughput etc
- Capacity requirements
- Must be in measurable terms



Design Constraints

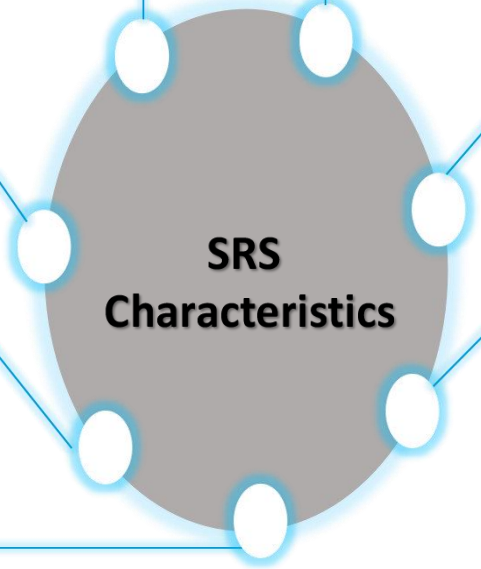
- Factors in the client environment that restrict the choices
- Some such restrictions
  - Standard compliance and compatibility with other systems
  - Hardware Limitations
  - Reliability, fault tolerance, backup req.
  - Security

External Interface

- All interactions of the software with people, hardware, and sw
- User interface most important
- General requirements of “friendliness” should be avoided
- These should also be verifiable

The system shall allow users to make payments using credit cards, debit cards, and PayPal.

CHARACTERISTICS OF SRS



Correctness
- Each requirement accurately represents some desired feature in the final system

Completeness
- All desired features or characteristics are specified
- Completeness and correctness strongly related

Unambiguous
- Each req has exactly one meaning
- Without this errors will creep in
- Important as natural languages often used

Verifiability
- There must exist a cost effective way of checking if sw satisfies requirements

Consistent
two requirements don't contradict each other

Ranked for importance/stability
Needed for prioritizing in construction
To reduce risks due to changing requirements

Traceable
The origin of the req, and how the req relates to software elements can be determined

SRS CONTENTS

Microsoft Word
97 - 2003 Documen

Requirements Review

- ✓ SRS reviewed by a group of people
- ✓ Group: author, client, user, dev team rep.
- ✓ Must include client and a user
- ✓ Process – standard inspection process
- ✓ Effectiveness - can catch 40-80% of req. errors

REQUIREMENTS IN AGILE

REQUIREMENTS IN AGILE

- Agile teams use **user stories** to capture user requirements in a simple, non-technical format.
- A user story is typically written in the form of:
  - As a [*type of user*], I want to [*perform some action*] so that [*some reason or benefit*].
  - As a *[user role], I* want *[requirement]* so that *[goal].*

Example:

- ✓ *As a customer, I want to save my payment information so that I can complete future purchases faster.*
- ✓ As an *online gamer*, I want to *see health information all the time on the screen* so that I *conveniently monitor my status.*
- ✓ As a *passenger*, I want to *see the driver's phone number* so that *I can call or text him/her in case we don't find one another*.
- ✓ As a *social media user*, I want to *add a picture to my profile* so that *other users know what I look like*.
- ✓ As a *credit card holder*, I want to *see the list of recent transactions* so that *I control my spending*.

How to write user stories?

- ✓ Ron Jeffries, the founder of Extreme Programming, proposed a so-called 3 C formula, describing the three essential components of user stories: Card, Conversation, and Confirmation.

Card

- ✓ Stories are traditionally written on note cards.
- ✓ Cards may be annotated with estimates, notes, etc.

Conversation

- ✓ Details behind the story come out during conversations with product owner

Confirmation

- ✓ Acceptance tests confirm a story was coded correctly.

BREAKING DOWN LARGE USER STORIES

- ✓ **Strategy 1:** Breaking down by workflow steps

As *customer,* I want to *pay for the goods in my shopping basket* so that I *receive my products at home*

Given a fairly standard order procedure, we could identify the following steps:

- ✓ *As customer I want to log-in with my account, so I don't have to re-enter my personal - information every time;*
- ✓ *As customer I want to review and confirm my order, so I can correct mistakes before I pay;*
- ✓ *As customer I want to pay for my order with a wire transfer, so that I can confirm my order;*
- ✓ *As customer I can pay for my order with credit card, so that I can confirm my order;*
- ✓ *As customer I want to receive a confirmation email with my order, so I have proof of my purchase;*

BREAKING DOWN LARGE USER STORIES

- ✓ **Strategy 2:** Breaking down by business rules

As *customer,* I want to *pay for the goods in my shopping basket* so that I *receive my products at home*

Given a fairly standard order procedure, we could identify the following 'business rules':

- ✓ *As shop owner I* want to *decline orders below 2 dollars, because they aren't profitable;*
- ✓ *As shop owner I* want to *decline customers from outside the US, because the shipping expenses make these orders unprofitable;*
- ✓ *As shop owner I* want to *reserve ordered products from stock for 48 hours, so other customers see a realistic stock;*
- ✓ *As shop owner I* want to *automatically cancel orders for which I have not received payment within 48 hours, so I can sell them again to other customers;*

BREAKING DOWN LARGE USER STORIES

- ✓ **Strategy 3:** Breaking down by happy / unhappy flow

As *customer,* I want to *log in with my account* so that I *I can access secured pages.*

If we consider a regular login procedure, we can identify a happy flow and several potential unhappy flows:

- ✓ As user I want to log in with my account, so that I can access secure pages (happy);
- ✓ As user I want to reset my password when my login fails, so I can try to log in again (unhappy);
- ✓ As user I want to have the option to register a new account if my login is not known, so I can gain access to secure pages (unhappy);
- ✓ As site owner I want to block users that log in incorrectly three times in a row, so I can protect the site against hackers (unhappy);

BREAKING DOWN LARGE USER STORIES

- ✓ **Strategy 4:** Breaking down by input options / platform

As *team member* I want to *view the Scrum Board*, so I *know how we're doing as a team*

We can identify the following input options:

- ✓ As team member I want to view the Scrum Board on my desktop, so I know the status of the sprint;
- ✓ As team member I want to view the Scrum Board on my mobile phone, so I know the status of the sprint;
- ✓ As team member I want to view the Scrum Board on a touchscreen, so I know the status of the sprint;
- ✓ As team member I want to view the Scrum Board on a print-out, so I know the status of the sprint;

- ✓ **Strategy 5:** Breaking down by data types or parameters

As *customer* I want to *search for products* so I can *view and order them*

- ✓ As customer I want to search for a product by its product number, so I can quickly find a product that I already know;
- ✓ As customer I want to search for products in a price range, so that the search results are more relevant;
- ✓ As customer I want to search for products by their color, so that the search results are more relevant;
- ✓ As customer I want to search for products in a product group, so that the search results are more relevant;

BREAKING DOWN LARGE USER STORIES

- ✓ **Strategy 6:** Breaking down by operations

As *shop owner* want to *manage products in my webshop*, so I can *update price and product information if it is changed*

We can identify the following input options:

- ✓ As shop owner I want to add new products, so customers can purchase them;
- ✓ As shop owner I want to update existing products, so I can adjust for changes in pricing or product information;
- ✓ As shop owner I want to delete products, so I can remove products that I no longer stock;
- ✓ As shop owner I want to hide products, so they cannot be sold for the time being;

- ✓ **Strategy 7:** Breaking down by test scenario / test case

As *manager* I want to *assign tasks to employees*, so they can *work on tasks*

- ✓ Test case 1: If an employee is already assigned, he or she cannot be assigned to another task;
- ✓ Test case 2: If an employee has reported in sick, he or she should be visually marked;
- ✓ Test case 3: If an employee has reported in sick, he or she cannot be assigned to a task;
- ✓ Test case 4: If an employee is not yet assigned, they can be assigned to a task;
- ✓ Test case 5: Employees can be assigned with a touchscreen monitor;

BREAKING DOWN LARGE USER STORIES

- ✓ **Strategy 8:** Breaking down by roles

As *news organization* we want to *publish new articles on our homepage*, so customers *have a reason to return periodically.*

If we consider a regular login procedure, we can identify a happy flow and several potential unhappy flows:

- ✓ As customer I want to read a new article, so I can be informed of important events;
- ✓ As journalist I want to write an article, so it can be read by our customers;
- ✓ As editor I want to review the article before putting it on the site, so that we can prevent typos;
- ✓ As admin I want to remove articles from the site, so that we can pull offending articles;
- ✓ As customer I want to review and confirm my order, so I can correct mistakes before I pay;

How to Calculate Story Points for User Stories

Step 1: Understand the overall effort involved in each user story

- ✓ *This involves discussing the completion goals, prep requirements, and any potential challenges associated with the story*

Step 2: Choose a baseline story

- ✓ *Select a simple, well-understood user story as a baseline or reference point. Each baseline story can represent a different task category or level of complexity, providing a framework for estimation.*

Step 3: Determine the sequencing method before assigning the actual numerical value

- ✓ *Many teams use a Fibonacci sequence for assigning story points, where each point represents the sum of the two numbers before it.*

Step 4: Register team consensus

- ✓ *Use a consensus-based estimation technique—like Planning Poker.*
- ✓ *In this method, a senior team member uses a deck of cards with numbers representing story points, and each junior team member selects a card that represents their estimate for the story*
- ✓ *All team members simultaneously reveal their chosen cards. If there is a significant variance in the estimates, the participants discuss their reasoning, after which they revise their estimates and select new cards.*
- ✓ *This process is repeated until the team reaches a consensus or a close approximation.*

Step 5: Record story point estimations

Step 6: Refine story point estimation with experience

Next Class

User interface Design